Disciplina: Teste de Software

Detecção de Bad Smells e Refatoração Segura

Nome do Aluno: Gabriel Assis de Souza

Matrícula: 863465

Repositório: github.com/GabriAssiss/bad-smells-js-refactoring

1. Análise de Smells (manual)

Analisando `src/ReportGenerator.js` identifiquei três Bad Smells:

1) Método Longo / Complexidade Ciclomática e Cognitiva alta. `generateReport` concentra cabeçalho, filtragem por perfil, formatação e rodapé em um único método com `if/else` aninhados em até quatro níveis. É difícil de ler, e como cada caminho (2 formatos × 2 perfis × prioridade) vive no mesmo método, qualquer mudança arrisca quebrar outro caminho e exige muitos casos de teste para cobrir todas as ramificações.

2) Código Duplicado. As linhas de formatação CSV/HTML e o `total += item.value` aparecem repetidos nos blocos ADMIN e USER. Alterar o formato de uma linha exige lembrar de editar todas as cópias; esquecer uma gera inconsistência entre perfis. Isso foi evidenciado na prática: o HTML do admin gerava `<tr >` (espaço extra) e fez um teste falhar.

3) Números Mágicos e Condicionais por tipo (strings soltas). Os valores `1000` e `500`, e as strings `'CSV'`, `'HTML'`, `'ADMIN'`, `'USER'`, estão espalhados no código sem nome. Não expressam a regra de negócio e adicionar um novo formato ou perfil obriga a modificar o método central (violação do Aberto/Fechado), dificultando também a testabilidade isolada das regras.

Outro ponto: o método muta os itens recebidos (`item.priority = true`), um efeito colateral que dificulta testes e raciocínio; mantive-o para preservar o comportamento.

2. Relatório da Ferramenta

Executado `npx eslint src/` com `eslint-plugin-sonarjs` (config recomendada, `cognitive-complexity` com limite 5 conforme o `.eslintrc.json` do repositório) antes da refatoração:

```
EOF
cat $S/eslint_before.txt | grep -v "^npm notice" >> $S/r.html
cat >> $S/r.html <<'EOF'
```

A ferramenta mediu a complexidade cognitiva do método em **27** (limite 5) — um número que a leitura manual só permite estimar de forma subjetiva, pois a métrica penaliza o aninhamento progressivamente. Também apontou o `no-collapsible-if` (linha 43), um `if` aninhado fundível que passou despercebido na análise manual. Em contrapartida, a análise manual encontrou smells que o linter não reporta: números mágicos, a duplicação semântica entre blocos e o efeito colateral sobre os itens. As duas abordagens se complementam.

3. Processo de Refatoração

Smell mais crítico: o método longo com condicionais aninhadas (complexidade 27). Criei `src/ReportGenerator.refactored.js` e uma cópia dos testes alterando apenas o import.

Antes (trecho) **Depois (trecho)**

```
for (const item of items) {
  if (user.role === 'ADMIN') {
    if (item.value > 1000) {
      item.priority = true;
    }
    if (reportType === 'CSV') {
      report += `${item.id},...\n`;
      total += item.value;
    } else if (reportType === 'HTML') {
      ...
      total += item.value;
    }
  } else if (user.role === 'USER') {
    if (item.value <= 500) {
      if (reportType === 'CSV') { ... }
      else if (reportType === 'HTML') {
... }
    }
  }
}
```

```
generateReport(reportType, user, items) {
  const formatter = FORMATTERS[reportType]
    ?? NULL_FORMATTER;
  const rule = ROLE_RULES[user.role]
    ?? NO_ACCESS_RULE;

  const visible =
    this.selectVisibleItems(items, rule);
  const total = this.sumValues(visible);

  const body = visible
    .map((i) => formatter.row(i, user))
    .join('');
  return (formatter.header(user) + body
    + formatter.footer(total)).trim();
}
```

Técnicas aplicadas: *Extract Method* (`selectVisibleItems`, `sumValues`); *Replace Conditional with Polymorphism* (estratégias em tabelas `FORMATTERS` e `ROLE_RULES` no lugar dos `if/else` por tipo e perfil, com objetos *Null Object* para valores desconhecidos, preservando o comportamento original); *Replace Magic Number with Symbolic Constant* (`ADMIN_PRIORITY_THRESHOLD`, `USER_MAX_VALUE`); eliminação de duplicação (cada formato de linha e a soma existem em um único lugar). Novos formatos ou perfis agora são adicionados sem alterar `generateReport`.

Validação:

```
$ npm test
PASS tests/ReportGenerator.test.js
PASS tests/ReportGenerator.refactored.test.js
Tests: 10 passed, 10 total

$ npx eslint src/ReportGenerator.refactored.js
(sem erros)
```

Ajustes necessários no projeto base (sem alterar os testes): (a) o script `test` passou a usar `NODE_OPTIONS=--experimental-vm-modules`, pois o projeto usa ES Modules e o Jest falhava com "Cannot use import statement outside a module"; (b) no arquivo original, corrigi o bug `<tr >` → `<tr>` que fazia um dos testes falhar, para que ambas as versões passem 100%.

4. Conclusão

A suíte de testes foi a rede de segurança que tornou a refatoração viável: como ela valida o comportamento observável e não a implementação, pude reestruturar totalmente o código e rodar os testes a cada passo, detectando de imediato regressões. Ela também revelou um bug latente no código original. Reduzir os smells (complexidade de 27 para abaixo do limite, sem duplicação e sem números mágicos) deixa o código mais legível, mais fácil de estender e de testar isoladamente, reduzindo o custo e o risco de manutenção futura. A análise estática com SonarJS complementa a revisão manual ao fornecer métricas objetivas e repetíveis, podendo ser integrada ao pipeline de CI.